NC STATE UNIVERSITY

Parallel Link-Based Ranking Algorithms

- Hyperlink Induced Topic Search Algorithm
- Pagerank

Sandhiya Shunmugavel and Sanjaey Shunmuga Sundaram

NC STATE UNIVERSITY

Why Link-Based Ranking ?

Ability to efficiently retrieve and rank relevant information is crucial

Enhances search result relevance using hyperlink structure

Complements content-based methods for better accuracy

Parallelizing Link Based Ranking Algorithms for Fast Tracking

NC STATE UNIVERSITY

Need for Parallel Link Based Algorithm

- Web graphs contain millions to billions of nodes/links
- Sequential execution is time-consuming and resource-heavy
- Parallel computing enables scalability and faster convergence
- MPI & OpenMP allow efficient processing of large-scale data

NC STATE UNIVERSITY

Dataset Used :

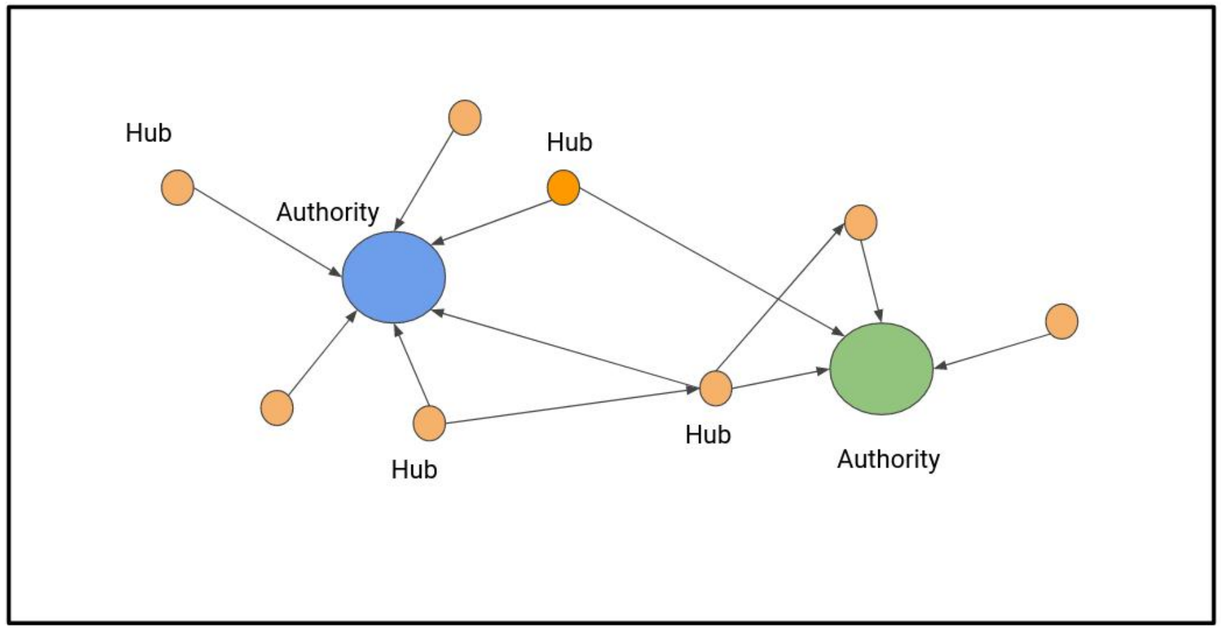
Dataset	Nodes	Edges	Type	Why Chosen for HITS/PageRank
web-Google	875,713	5,105,039	Directed Web Graph	Crawled web hyperlinks; ideal for PageRank (random surfer) and HITS (hub-authority).
wiki-Talk	2,394,385	5,021,410	Directed Communication Network	Captures real user-to-user interactions; suitable for influence flow and authority analysis.
Orkut Network	3,072,441	117,185,083	Undirected Social Graph	Very dense community structure; great for testing scalability and ranking in large social networks.
ca-GrQc	5,242	14,496	Undirected Collaboration	Co-authorship network with clear hub-authority structure; perfect for HITS demonstration.

NC STATE UNIVERSITY

Hyperlink Induced Topic Search Algorithm

- Web link structures to discover and rank the webpages relevant for a particular search.

A web page is named as HUB if the page point to various hyperlinks and as AUTHORITY if the web page is pointed by many hyperlinks.



NC STATE UNIVERSITY

Algorithm Steps

- Initialize the hub and authority of each node with a value of 1
- For each iteration, update the hub and authority of every node in the graph
- The new authority is the **sum of the hub** of its parents
- The new hub is the **sum of the authority** of its children
- Normalize the new authority and hub

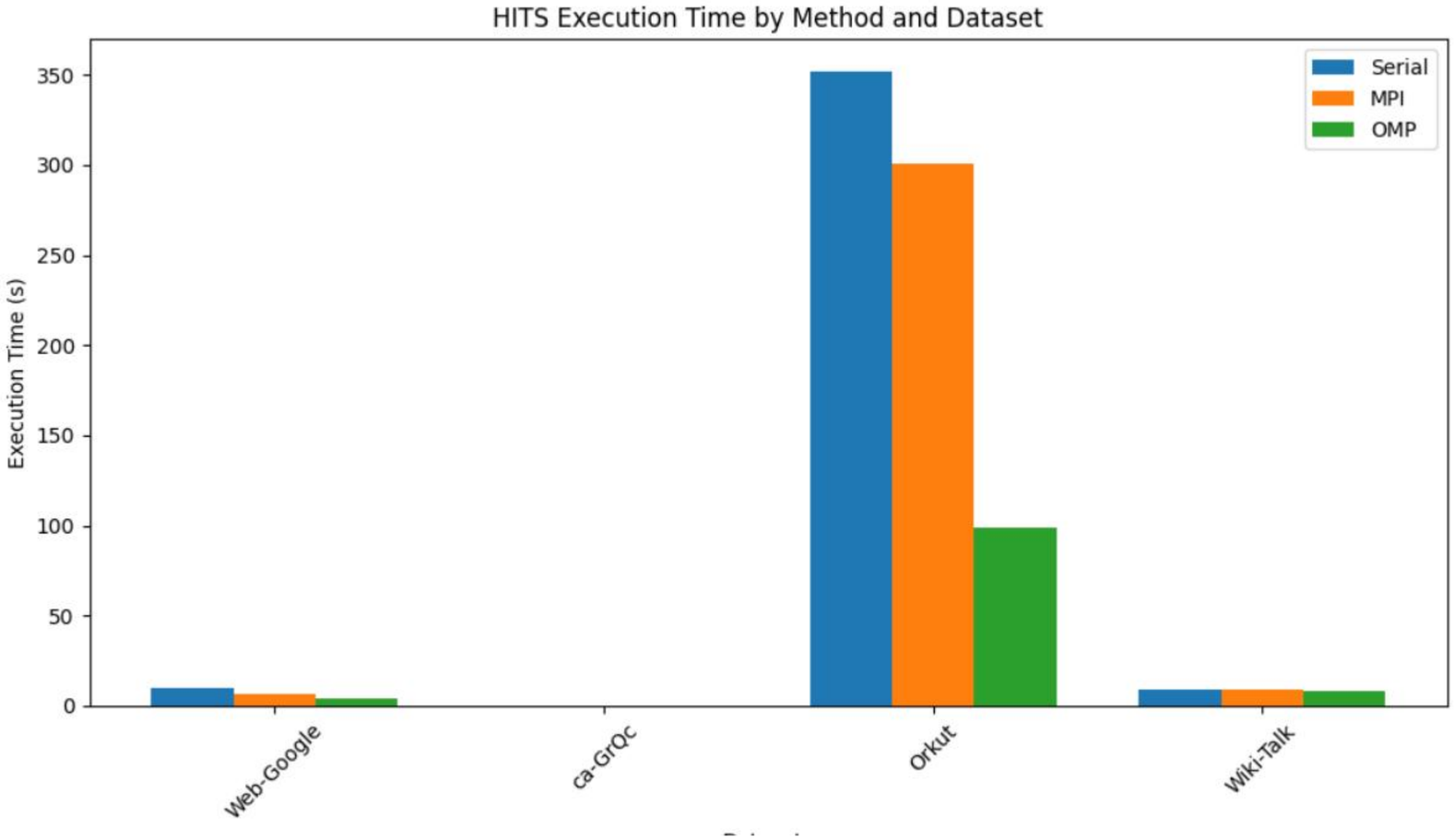
```
// Update authority scores: A = A^T * H
for (int i = 0; i < coo->num_nonzeros; ++i)
    new_auth[coo->cols[i]] += hub[coo->rows[i]];

// Normalize authority
normalize_vector(new_auth, N);

// Update hub scores: H = A * A
for (int i = 0; i < coo->num_nonzeros; ++i)
    new_hub[coo->rows[i]] += new_auth[coo->cols[i]];
```

NC STATE UNIVERSITY

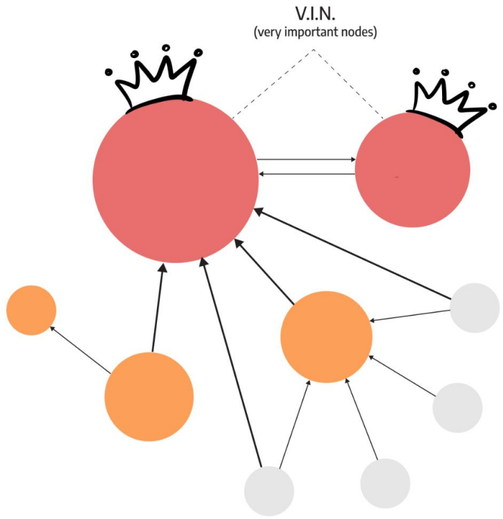
Parallelize HITS



NC STATE UNIVERSITY

Page Rank

- It was the original algorithm that Google search used to rank websites in their search engine results.
- Page Rank works by counting the number and quality of links to a page to determine a rough estimate of how important a particular website is.



NC STATE UNIVERSITY

Algorithm Steps

- Each node starts with equal rank.
- In each iteration, the rank is distributed from a page to its linked pages based on its out-degree.
- A damping factor simulates the probability of continuing to click links vs jumping to a random page.
- The process repeats until the PageRank values converge (i.e., changes become very small).

$$PR(P_i) = \frac{(d)}{n} + (1-d) \times \sum_{l_{j,i} \in E} PR(P_j) / \text{Outdegree}(P_j)$$

D(damping factor)=0.1~0.15
n=|page set|

NC STATE UNIVERSITY

Parallel Page Rank -MPI and OpenMP

MPI

OpenMP

```
1. Partition the graph:
   - chunk = N / size
   - start = rank * chunk
   - end = (rank == size - 1) ? N : start + chunk

2. For each iteration:
   a. Set new_pr[i] = (1 - d) / N  for all nodes in [start, end)

   b. For each edge (src, dst):
        If dst is in [start, end) and out_degree[src] > 0:
            contribution = d * pr[src] / out_degree[src]
            new_pr[dst] += contribution

   c. Use MPI_Allreduce to sum all new_pr[] vectors → global_pr[]

   d. Compute convergence:
      local_diff = sum(|global_pr[i] - pr[i]|) for i in [0, N)
      MPI_Allreduce to get global_diff
```

```
Input: Graph in COO format (rows[], cols[]), damping factor d, max iterations

1. Initialize out_degree[] = 0
   Parallel for each nonzero edge:
      Atomically increment out_degree[source]

2. Initialize pr[] = 1 / N for all N nodes

3. For each iteration up to max_iter:
    a. Initialize new_pr[] = (1 - d) / N
       (Uniform teleportation value)

    b. Parallel for each edge (source, dest):
          If out_degree[source] > 0:
             contribution = d * pr[source] / out_degree[source]
             Atomically add contribution to new_pr[dest]

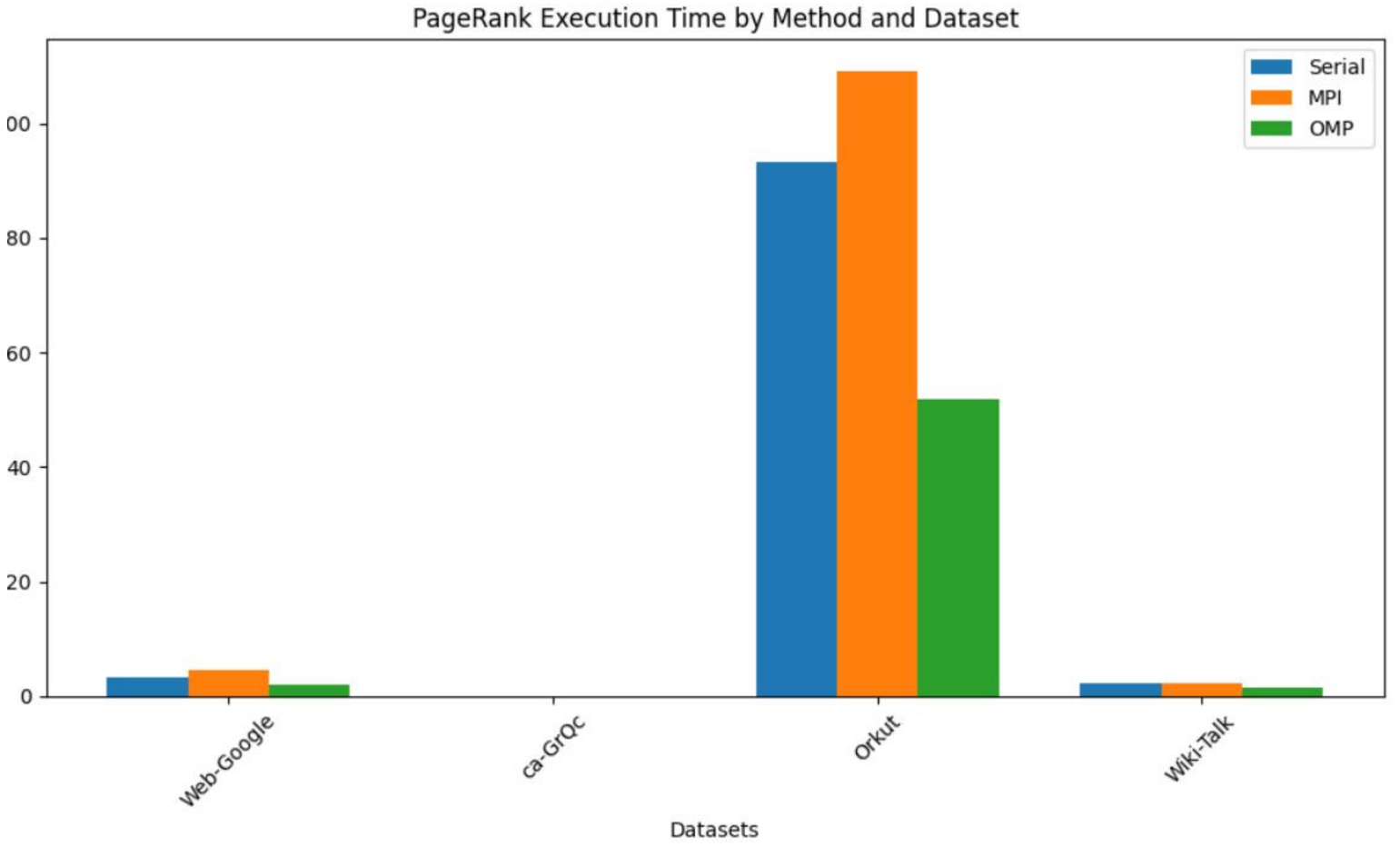
    c. Compute diff = sum of absolute differences between pr and new_pr
       (Use parallel reduction)

    d. If diff < EPSILON:
          Converged → break loop

    e. Swap pr and new_pr for the next iteration
```

NC STATE UNIVERSITY

Parallelize Page Rank



NC STATE UNIVERSITY

Main Takeaways and Next Steps

Takeaways

1. Demonstrated **performance gains** through parallelization using **MPI and OpenMP**
2. **Execution time reduced** significantly compared to serial versions
3. Reinforced the importance of **parallel computing** in handling web-scale data

Next Steps

1. **Implement GPU-based acceleration** using CUDA for even faster performance
2. **Scale to real-world web graphs** (e.g., Common Crawl, Wikipedia dumps)

NC STATE UNIVERSITY

References

1. Authoritative Sources in a Hyperlinked Environment
2. https://en.wikipedia.org/wiki/HITS_algorithm
3. The Pagerank Citation Ranking : Bringing Order to the Web